

第四章 汇编语言编程

Dr. Prof. Ni Li

4.1 汇编语言编程

汇编语言

- Assembly language
 - 指令助记符, 符号地址（变量），地址标号
 - MOV, ADD, AND, CMPSB,
 - X DB 'HELLO'
 - Next: MOV AL, BL
 - Source code: 指令instructions + 伪指令pseudo-instructions
 - 伪指令：指明程序汇编、链接、加载过程; 不生成机器码

- 机器语言

machine language program

0000 B0 09

0002 04 08

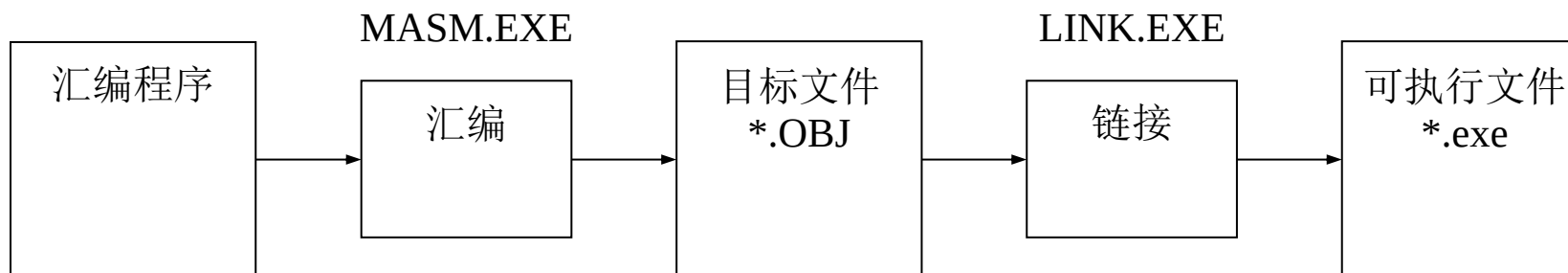
assembly language program

MOV AL, 9

ADD AL, 8

Program execution

- **MASM: 8086系统的汇编器**
 - 汇编: 检查语法错误, 生成二进制目标文件
 - 链接: 从多个目标文件、库文件中生成可执行文件



- **程序错误**
 - 语法错误—>MASM
 - 执行逻辑错误—>Debug

Hello world demo

Program execution

- 1. 汇编语言源码
 - 文本编辑器: .ASM
- 2. 汇编
 - 把源码文件转换成二进制代码: .ASM→.OBJ
 - C:\>**MASM** file-name[.ASM];
 - .lst list file: 源程序和目标程序的对照
- 3. 链接
 - .OBJ→.EXE
 - C:\>**LINK** file-name;
- 4. 执行
 - C:\> file-name

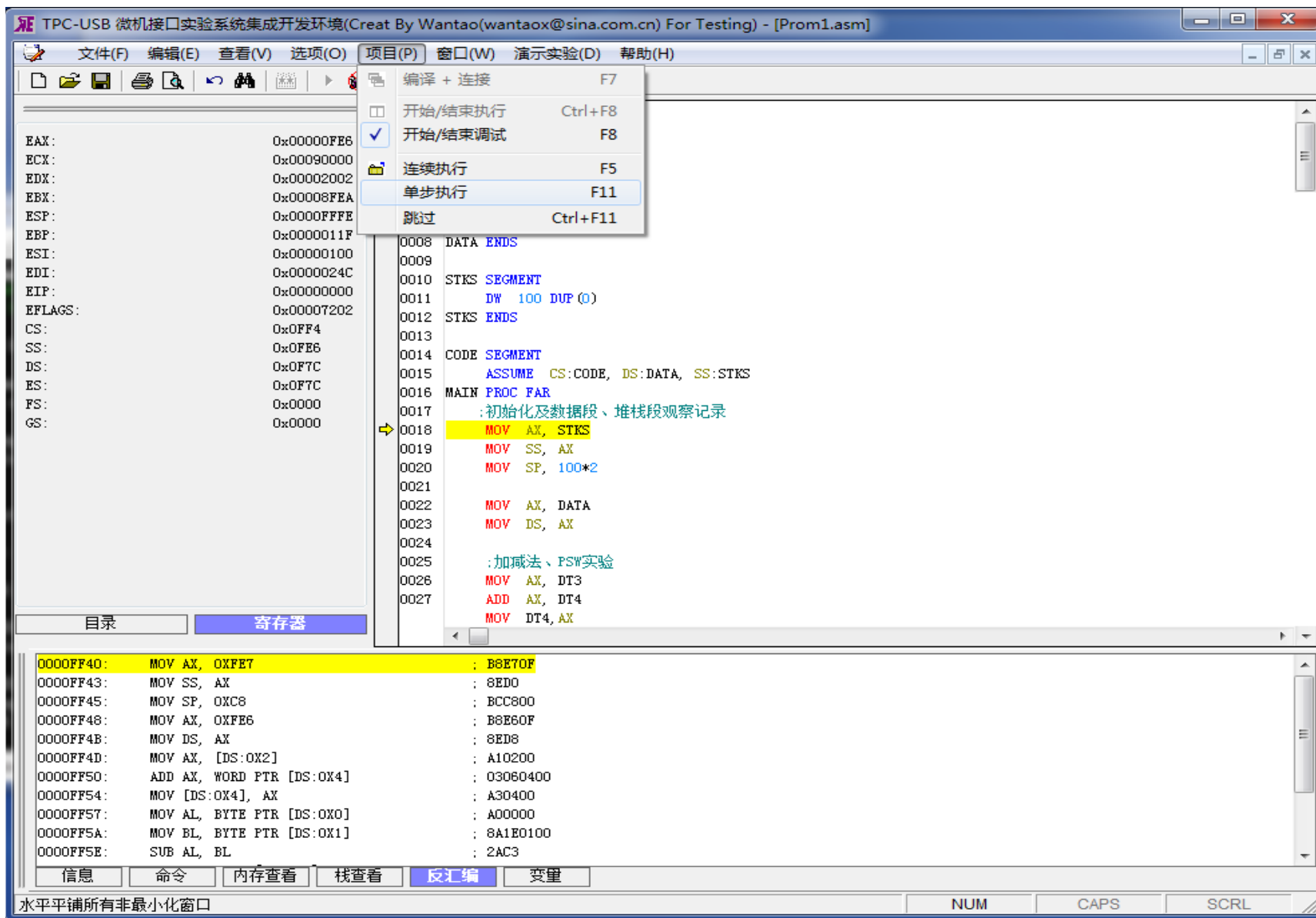
0000 B0 09

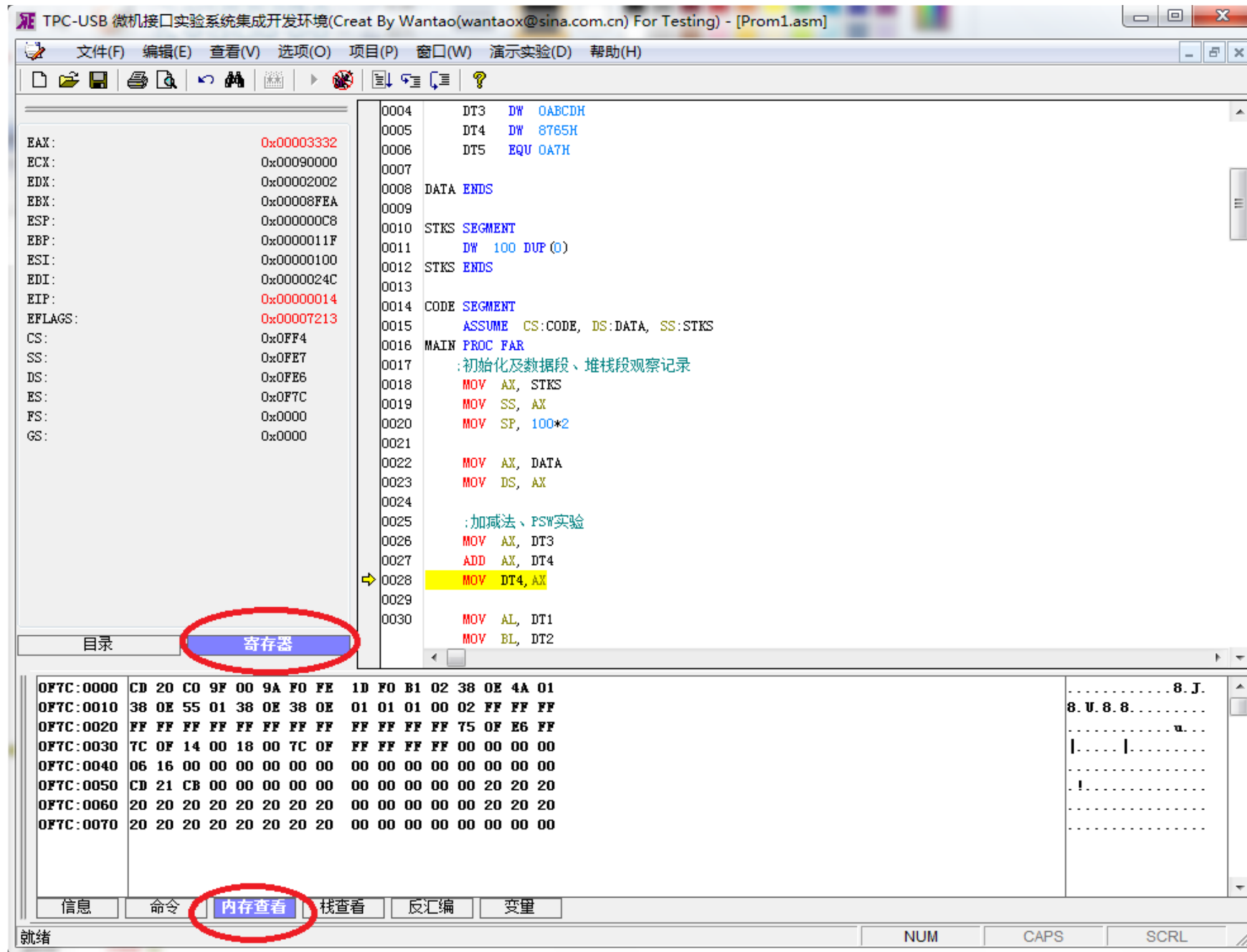
MOV AL, 9

0002 04 08

ADD AL, 8

64-bit OS





Examples: 显示 “HELLO World! ”

```
STACK SEGMENT STACK
    DW 100 DUP (0)
```

```
STACK ENDS
```

```
DATA SEGMENT
```

```
    X DB 'HELLO World!',0DH,0AH,'$'
```

```
DATA ENDS
```

```
CODE SEGMENT
```

```
    ASSUME CS:CODE, DS:DATA, SS:STACK
```

```
MAIN PROC FAR
```

```
START: PUSH DS
```

```
    XOR AX,AX
```

```
    PUSH AX
```

```
    MOV AX, DATA
```

```
    MOV DS, AX
```

```
    MOV DX, OFFSET X
```

```
    MOV AH, 9
```

```
    INT 21H
```

```
    RET
```

```
MAIN ENDP
```

```
CODE ENDS
```

```
END MAIN
```

堆栈段

数据段

指令

代码段

- 由不同段组成
 - 段定义伪指令
- 指令 + 伪指令

汇编程序

```
STACK SEGMENT STACK
    DW 100 DUP (0)
STACK ENDS
```

- Stack segment
 - 定义堆栈段的**两种方式**
 - 声明作为堆栈段使用
 - **STACK** SEGMENT **STACK**
 - 在代码段中设置SS:SP

```
MOV AX, STACK
MOV SS, AX
MOV SP, 200
```

汇编程序

- Data segment
 - 存储程序处理所需数据
 - 根据访问数据的需求来定义数据段

```
DATA SEGMENT
    X DB 'HELLO World!',0DH,0AH,'$'
DATA ENDS
```

汇编程序

- Code segment
 - 至少有一个代码段
 - **ASSUME** 语句: ASSUME (代码段特殊, 代码段的段地址自动放入CS寄存器)
 - 多个程序(procedures): 程序定义, 主程序, 子程序
 - INT 21H function 9
 - DS:DX
 - '\$'
 - AH: 9

DATA SEGMENT

X DB 'HELLO World!',0DH,0AH,'\$'

DATA ENDS

CODE SEGMENT

ASSUME CS:CODE, DS:DATA, SS:STACK

MAIN PROC **FAR**

START: PUSH DS

XOR AX,AX

PUSH AX

MOV AX, DATA

MOV DS, AX

MOV DX, OFFSET X

MOV AH, 9

INT 21H

RET

MAIN ENDP

CODE ENDS

汇编程序

- End
 - END + 主程序名称 或 起始指令地址标号
 - END MAIN
 - END START

MAIN PROC FAR

START: PUSH DS

XOR AX,AX

PUSH AX

MOV AX, DATA

MOV DS, AX

MOV DX, OFFSET X

MOV AH, 9

INT 21H

RET

MAIN ENDP

CODE ENDS

END MAIN

汇编程序

- Program Segment Prefix(PSP)

程序段前缀

- 256 字节信息, 由操作系统加载
- 用于程序控制和管理
- PSP起始两个字节

00-01H:INT 20H

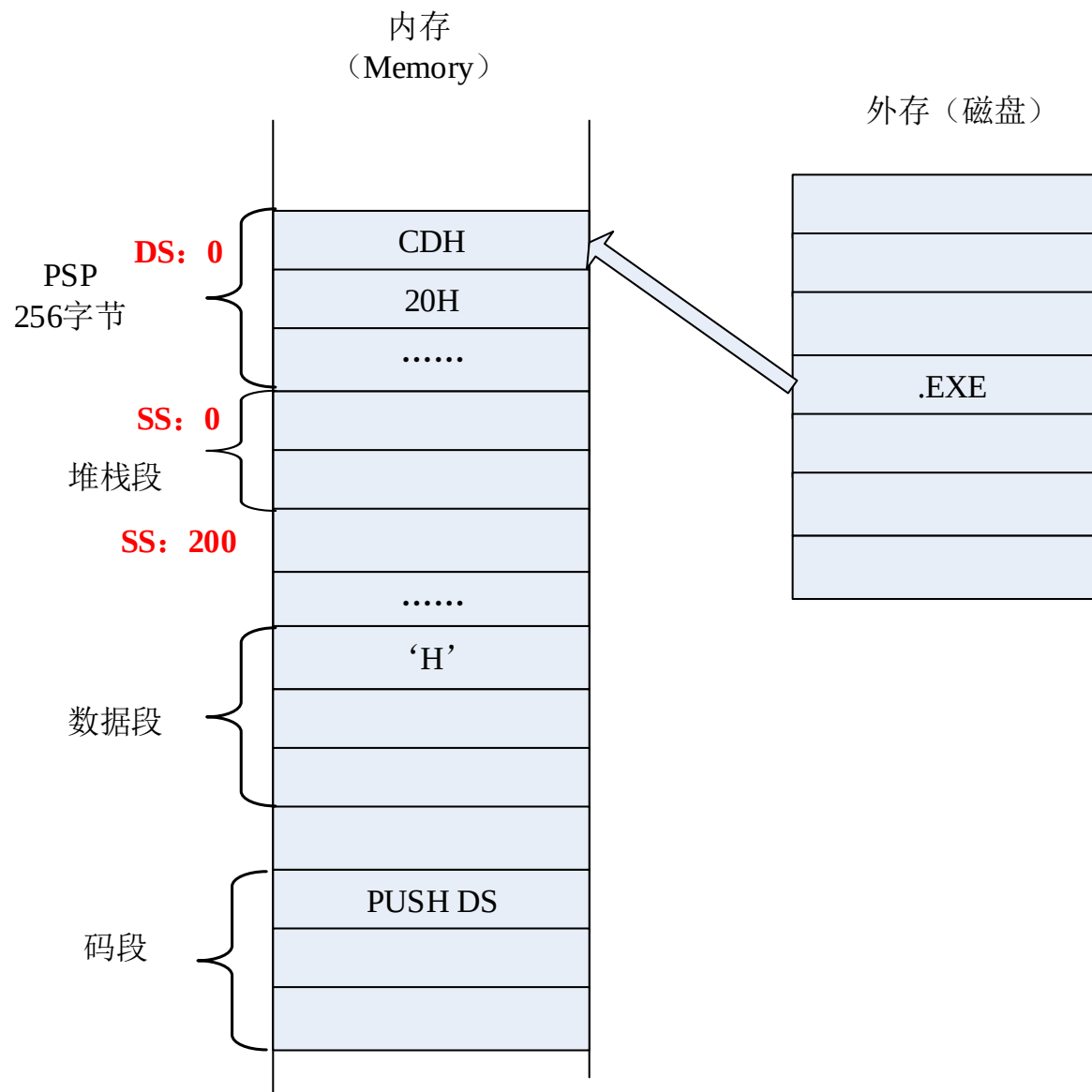
```
0000                                STACK SEGMENT STACK
0000 0064 [                          ST1 DB 100 DUP(0)
      00
      ]
0064                                STACK ENDS
-----
0000                                DATA SEGMENT
0000 48 65 6C 6C 6F 20              X DB 'Hello World!', 0DH, 0AH, '$'
      57 6F 72 6C 64 21
      0D 0A 24
000F                                DATA ENDS
-----
0000                                CODE SEGMENT
                                ASSUME CS:CODE, DS:DATA, SS:STACK
0000                                MAIN PROC FAR
0000 1E                                NEXT: PUSH DS
0001 33 C0                            XOR AX, AX
0003 50                                PUSH AX
0004 B8 ---- R                        MOV AX, DATA
0007 8E D8                            MOV DS, AX
0009 B8 ---- R                        MOV AX, STACK
000C 8E D0                            MOV SS, AX

000E BA 0000 R                        MOV DX, OFFSET X
0011 B4 09                            MOV AH, 9
0013 CD 21                            INT 21H

0015 CB                                RET
0016                                MAIN ENDP
0016                                CODE ENDS
-----
                                END MAIN
```

汇编程序

- 可执行程序在内存中的存储



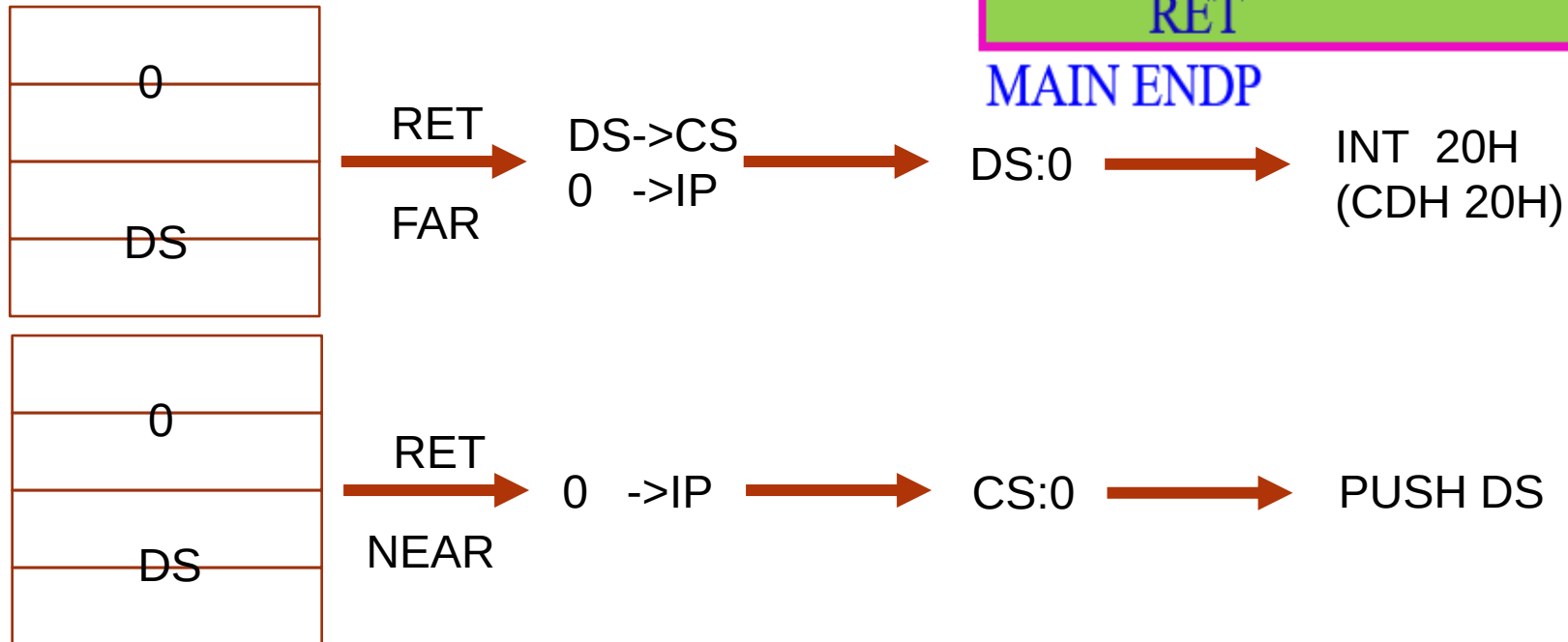
汇编程序

- 利用 PSP 退出程序返回操作系统 (单任务OS)
 - Q: 近调用?
- INT 21H function 4CH

MAIN PROC FAR

```
START: PUSH DS
      XOR AX,AX
      PUSH AX
      MOV AX, DATA
      MOV DS, AX
      MOV DX, OFFSET X
      MOV AH, 9
      INT 21H
      RET
```

MAIN ENDP



汇编程序

- 汇编程序组成
 - 段定义
 - 段分配(Assume 代码段自动分配)
 - 设置段地址(Data, Stack)
 - 程序逻辑(functions)
 - 返回DOS
 - End语句（主程序名称或者主程序起始地址标号）
 - 需要时调用子程序

4.2 MASM 表达式

与指令不同

- 区别:
 - MASM 表达式: 汇编时计算
 - 指令: 执行程序时计算
- 分类
 - Arithmetic, logic, return value...

算术操作符

- Arithmetic operation
 - +, -, *, /, SHL, SHR
 - 操作对象和结果: integer

- E.g.

数据段:

```
ARRAY DB 1,2,3,4,5,6,7,8
```

```
TRY DB 20
```

...

码段:

```
MOV AX, 30*5
```

```
MOV CX, (TRY-ARRAY)
```

汇编器计算并生成指令:

```
MOV AX, 150
```

```
MOV CX, 8
```

逻辑操作符

- 逻辑操作
 - AND, OR, XOR, NOT

- E.g.

```
MOV  AL, NOT 0FFH
MOV  BL, 8CH AND 73H
MOV  AH, 8CH OR 73H
MOV  CH, 8CH XOR 73H
```

汇编器计算并生成指令:

```
MOV  AL, 0
MOV  BL, 0
MOV  AH, 0FFH
MOV  CH, 0FFH
```

关系操作符

- EQ, NE, LT, LE, GT, GE
- 结果:
 - 输出的每一个二进制位为**1**: 结果为真
 - 输出的每一个二进制位为**0**: 结果为假
- E.g.

MOV AX, 10H GT 16

MOV BL, 6 EQ 0110B

MOV AL, 0FFH LE 1

汇编器计算并生成指令

MOV AX, 0

MOV BL, **0FFH**

MOV AL, 0 无符号数比较

MASM 表达式

- TYPE: 返回标号的距离属性或者变量的类型属性

- E.g.

数据段:

```
A1 DB 20H
A2 DW 0438H
A3 DD ?
```

码段:

```
L1:  MOV AH, TYPE A1
      MOV BH, TYPE A2
      MOV AL, TYPE A3
      MOV BL, TYPE L1
```

	Type	Return
variable	DB	1
	DW	2
	DD	4
	DQ	8
	DT	10
label	NEAR	-1 (FFH)
	FAR	-2 (FEH)

汇编器计算并生成指令:

```
MOV AH, 1
MOV BH, 2
MOV AL, 4
MOV BL, 0FFH
```

MASM 表达式

- LENGTH

- 返回使用**DUP**定义时**DUP前面的单元数**，否则返回 **1**

- E.g.

M1	DW	100 DUP(?)
M2	DW	1,2,3
M3	DB	'ABCD'

MOV	CX, LENGTH M1
MOV	BL, LENGTH M2
MOV	AL, LENGTH M3

汇编器计算并生成指令:

MOV	CX, 100
MOV	BL, 1
MOV	AL, 1

MASM 表达式

- SIZE (SIZE = TYPE* LENGTH)

- 返回变量的所有字节数

- E.g.

M1 DW 100 DUP(?)

M2 DW 1,2,3

M3 DB 'ABCD'

MOV CX, SIZE M1

MOV BL, SIZE M2

MOV AL, SIZE M3

汇编器计算并生成指令:

MOV CX, 200

MOV BL, 2

MOV AL, 1

- SEG, OFFSET

4.3 伪指令

数据定义

- 格式
 - 变量名 D* 操作数, 操作数,
 - 变量名 D* n DUP(操作数, 操作数,
 - DB, DW, DD: 分配 1, 2, 4 个字节的内存单元
- 注意
 - DW可定义2个ASCII码
 - 超过2个时用DB定义
 - X DW 'AB' MOV AL, X ?
 - 使用? 留出存储空间
 - 使用 DUP 定义重复数据
 - 变量的定义必须在数据段内



DATA SEGMENT

X1 DW ?, 'AB', 1234H

DATA ENDS

数据定义

- DW
 - 存储变量或者标号的偏移地址到内存中
- DD
 - 存储变量或者标号的逻辑地址到内存中：低字节存储偏移地址，高字节存储段地址
- Examples

若有变量PAR1, 地址标号 ADR2, ADR3
且CS=2000H, PAR1, ADR2, ADR3的偏移
地址分别为: 0100H, 0200H, 0300H

```
ONE DW PAR1;  
TWO DW ADR2;  
THREE DD ADR3;
```



数据定义

- Example

\$: 表示当前段的偏移地址

DATA SEGMENT

X1 DW X1, 'AB' , 1234H, \$+2; 0, 4142H, 1234H, 8

DATA ENDS

CODE SEGMENT

.....

MOV BX, '\$'; ASCII码24H

MOV BX, \$; 当前代码段偏移地址

.....

表达式定义

- 格式
 - 符号名 EQU 值
 - 定义变量, 标号, 常数和指令
 - PURGE 取消定义
 - **Examples**

```
COUNT EQU 100  
MOV SI, COUNT
```

```
C1 EQU ADD  
C1 CX, AX  
PURGE COUNT
```

段定义

- 格式

段名 **SEGMENT** 定位类型 组合类型 ‘类型名称’

逻辑段内容

段名 **ENDS**

- 定位类型 **(diagram): 地址类型**

- PAGE: 1 PAGE=256 bytes 物理地址***00H
- PARA: 1 PARA=16 bytes 物理地址****0H **default**
- WORD: 1 WORD=2 bytes 偶地址
- BYTE: 从任意地址起

段定义

- 组合类型

- PUBLIC

- COMMON

- STACK

- NONE: default, placed separately

段分配

- 格式
 - ASSUME **CS**: 段名, DS:段名, SS:段名, ES:段名
 - 代码段必须进行分配声明
 - CS: 代码段的段地址**自动**装入CS寄存器
 - 定义了段, 都要进行分配声明

程序定义

- 程序定义 Procedure definition

- Format

MSUB

PROC FAR

... ..

CALL SUB1

- 程序调用 Procedure calling

- Format

... ..

RET

MSUB

ENDP

SUB1

PROC NEAR

... ..

RET

SUB1

ENDP

开始与结束伪指令

- **ORG**
 - 格式: ORG 表达式
 - 定义后续声明变量的起始偏移地址
 - 表达式必须是正整数

- **Examples**

```
DATA SEGMENT
    DA1 DB 11H;
    ORG $+100H;
    DA2 DB 22H;
DATA ENDS
```



4.4 DOS 功能调用

DOS 功能调用（输入输出）

- 1号功能

- 从标准输入设备读入一个字符(键盘)，并显示在标准输出设备上（显示器）
- 对应的 ASCII码存储在中AL
- 等待输入

```
MOV AH, 1  
INT 21H
```

- 2号功能

- 显示字符，对应的 ASCII码存储在DL中
- 实验室用的汇编环境INT 21H的2号功能调用会改变AL的值，在使用前后需要把AX进行出入栈保护

```
MOV DL, 'A'  
MOV AH, 2  
INT 21H
```

- 8号功能

- 与1号功能类似，只是不回显
- 等待输入

```
MOV AH, 8  
INT 21H
```

DOS 功能调用 (输入输出)

- 9号功能

- 显示字符串: DS:DX(字符串起始地址)
- 字符串以'\$'结尾

```
MOV AX, DATA
MOV DS, AX
MOV DX, OFFSET X
MOV AH, 9
INT 21H
```

- 6号功能

- 判断是否键盘有按键按下 DL=0FFH
- ZF=1, 无按键按下(JZ); 否则, 字符的ASCII 码存在 AL中
- 不等待键入

```
MOV DL, 0FFH
MOV AH, 6
INT 21H
JZ Next;(zf=1表示键盘无输入)
```

- 4CH号功能 : 返回DOS

```
MOV AH, 4CH
INT 21H
```

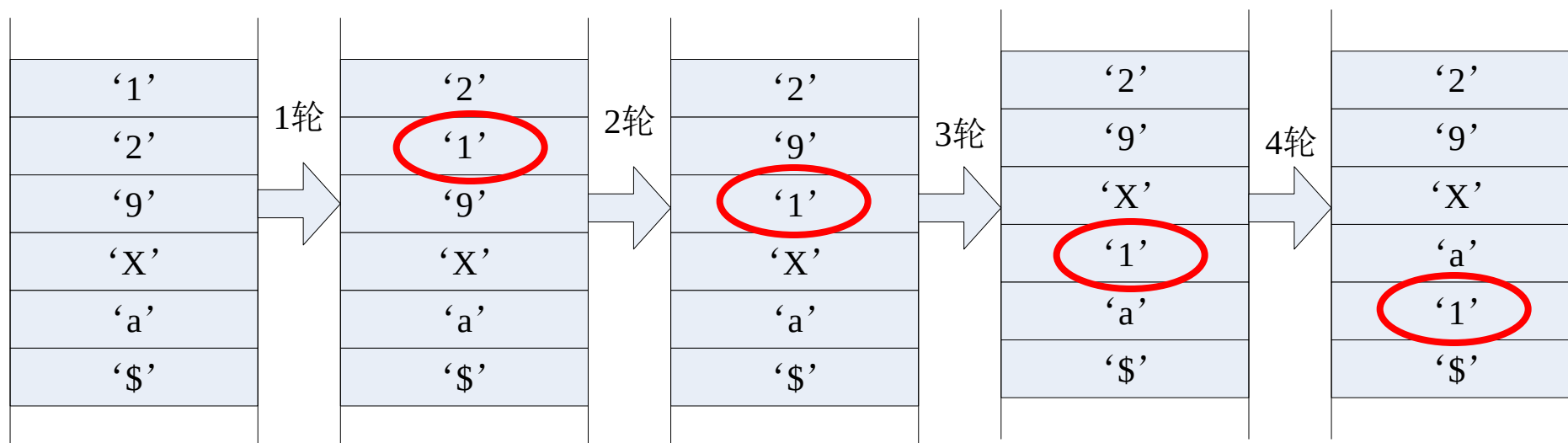
4.6 汇编语言编程

典型应用

- 字符串移动、比较和查找
 - 字节, 字, 有符号和无符号数
- 判断正负、奇偶
- 字符输入和显示(不同数制)

Examples

- 将字符串从大到小排序
- 排序方法：冒泡法
 - 循环比较次数



Examples

核心: 两两比较

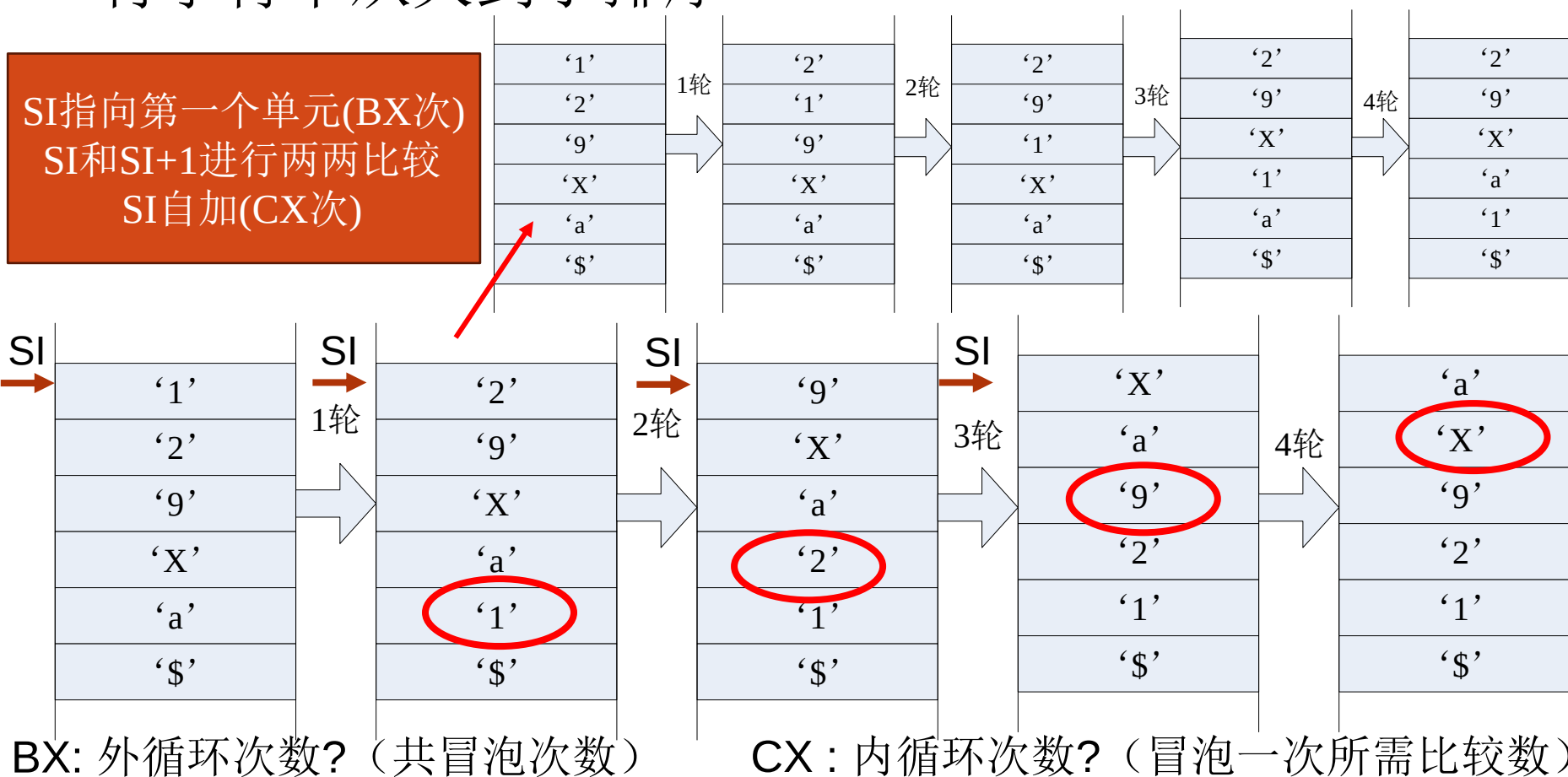
第一轮: SI指向第一个单元, 两两比较4次 (SI自加)

第二轮: SI指向第一个单元, 两两比较3次 (SI自加)

第三轮: SI指向第一个单元, 两两比较2次 (SI自加)

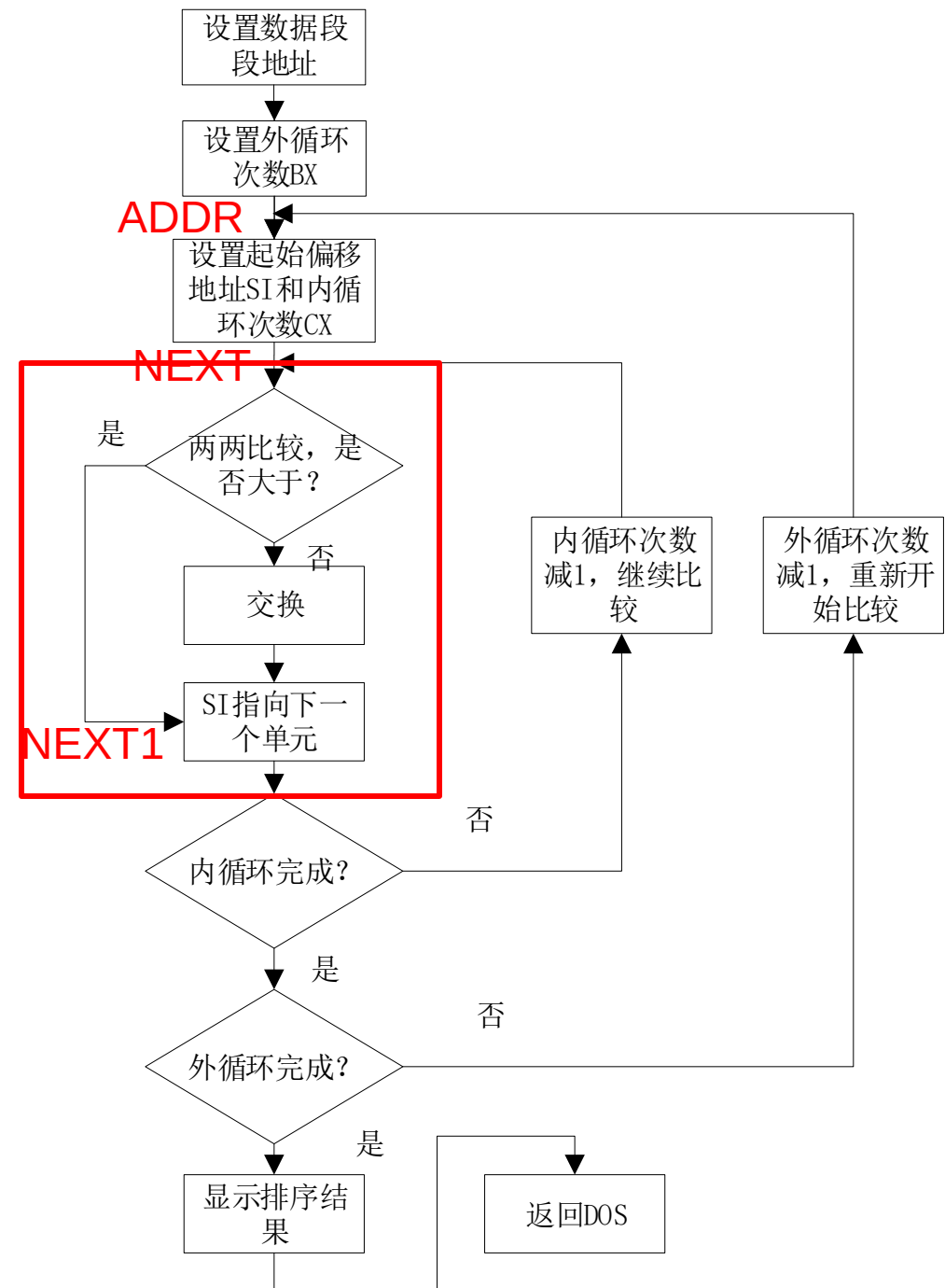
第四轮: SI指向第一个单元, 两两比较1次 (SI自加)

• 将字符串从大到小排序



Examples

- 从大到小排序
 - 冒泡法
 - 循环次数的确定



```

STACK SEGMENT STACK
    DB 100 DUP('STACK');
STACK ENDS; SP=500

DATA SEGMENT
    X1 DB '129Xa', '$'
;$为当前地址，$-X1-2为比较的轮数
    COUNT EQU $-X1-2;
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA,
SS:STACK
MAIN PROC
    MOV AX, DATA
    MOV DS, AX

```

```

MOV BX, COUNT; 外循环次数
ADDR:
MOV CX, BX ; 内循环次数与外循环相同
MOV SI, OFFSET X1
NEXT:
    MOV AL, [SI]; 内循环
    CMP AL, [SI+1]
    JAE NEXT1; 大于等于则继续比较
    XCHG AL, [SI+1]; 小于则交换
    MOV [SI], AL
NEXT1:
    INC SI; 地址加1
    LOOP NEXT
    DEC BX
    JNZ ADDR

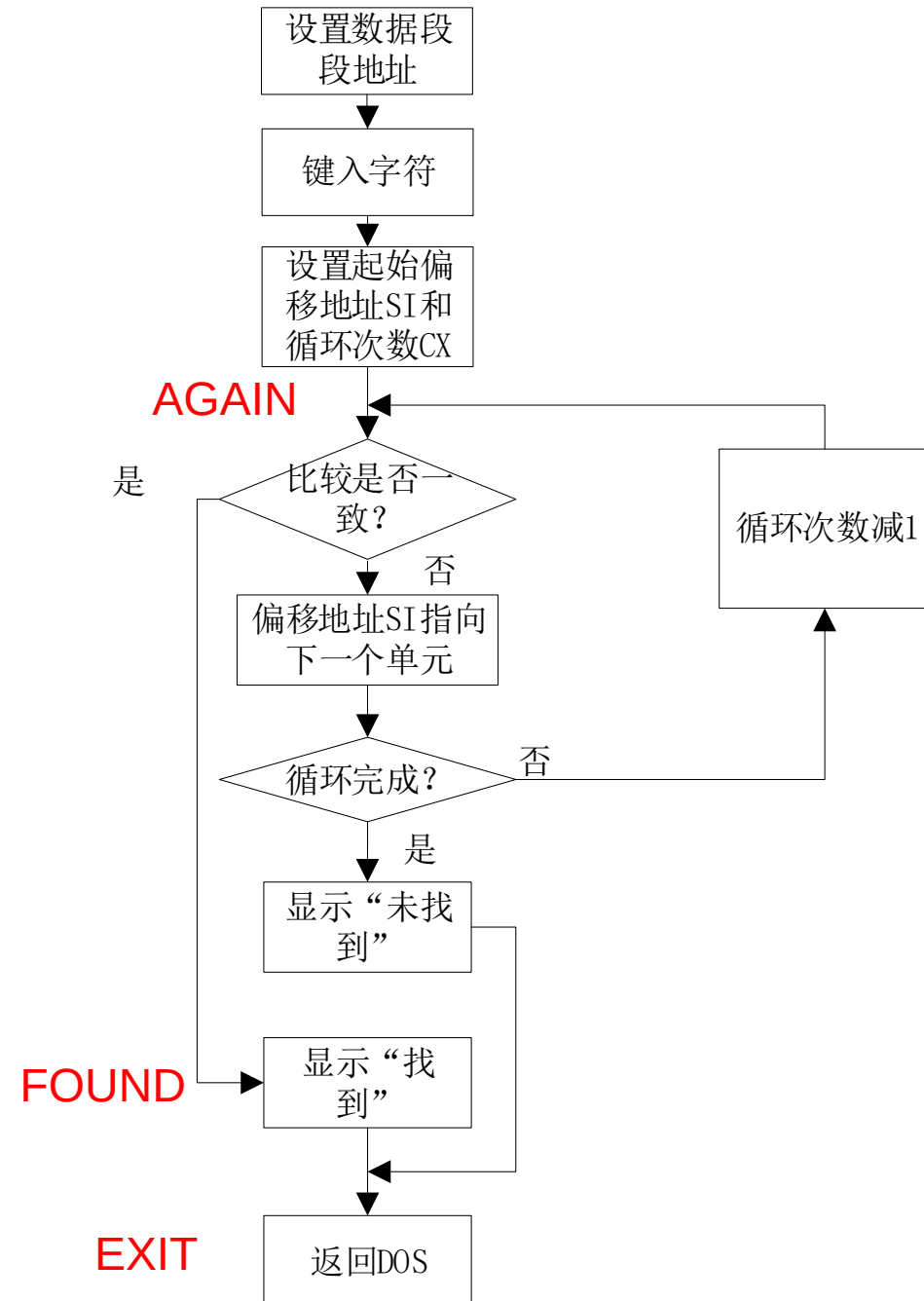
    MOV DX, OFFSET X1
    MOV AH, 9
    INT 21H
    MOV AH, 4CH
    INT 21H
MAIN ENDP
CODE ENDS
END MAIN

```

Examples

- 搜索数据段中是否存储键盘上键入的字符; 显示 “FOUND” or “NOT FOUND”

数据段寻址 DS:SI [SI]
CX LOOP
DEC JNZ



DATA SEGMENT

X1 DB 100 DUP (?)

X2 DB 'FOUND\$'

X3 DB 'NOT FOUND\$'

DATA ENDS

CODE SEGMENT

ASSUME CS:CODE, DS:DATA, SS:STACK

MAIN PROC

;将DATA的段地址给DS, **DS缺省对着程序段前缀**

MOV AX, DATA

MOV DS, AX;

MOV SI, OFFSET X1

MOV CX, **SIZE X1**; 作为LOOP的循环次数

MOV AH, 1

INT 21H; 1号功能为输入1个字符, 并存入AL中

AGAIN:

CMP AL, [SI]

JE FOUND

INC SI

LOOP AGAIN; CX自动减1

MOV DX, OFFSET X3

MOV AH, 9

INT 21H

JMP EXIT

FOUND:

MOV DX, OFFSET X2;

MOV AH, 9

INT 21H

EXIT:

MOV AH,4CH

INT 21H; 返回DOS

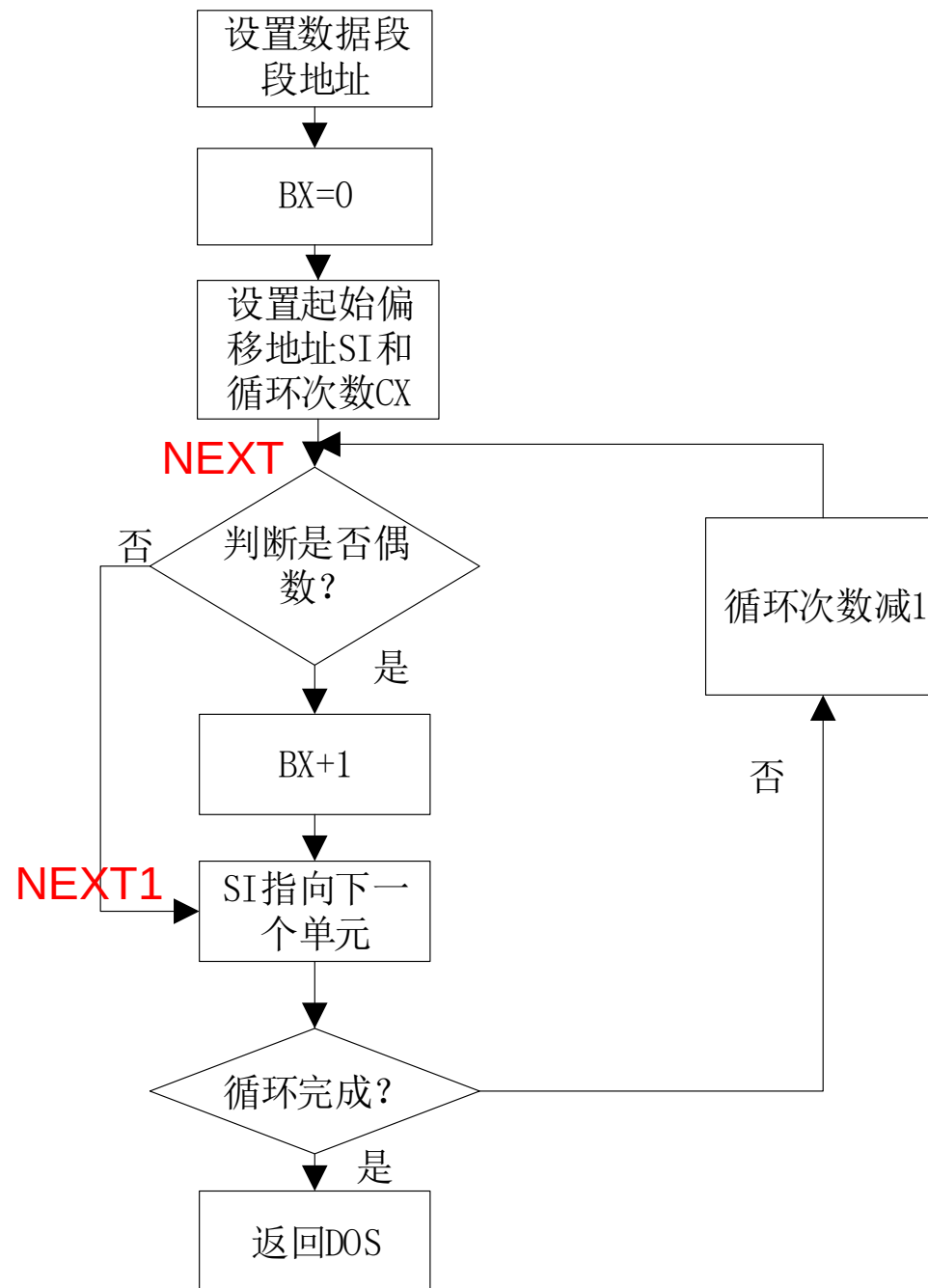
MAIN ENDP

CODE ENDS

END MAIN

Examples

- 找出数据段偶数的个数放入->BX



STACK SEGMENT STACK

DB 100 DUP(?)

STACK ENDS

DATA SEGMENT

X DB 100 DUP(?)

DATA ENDS

判正负: TEST AL, 80H

CODE SEGMENT

ASSUME CS:CODE, DS:DATA, SS:STACK

MAIN PROC FAR

PUSH DS

XOR AX, AX

PUSH AX

MOV AX, DATA

MOV DS, AX

MOV SI, OFFSET X

MOV CX, 100; 准备循环

MOV BX, 0; 偶数个数计数

NEXT:

MOV AL, [SI]

TEST AL, 1; 与1相与, 为0则偶, 为1则奇

JNZ NEXT1

INC BX; 偶数个数加1

NEXT1:

INC SI; 地址自加

LOOP NEXT

RET

MAIN ENDP

CODE ENDS

END MAIN

Examples

- **X DB 0AAH**
 - 按照二进制显示 10101010B
 - 按照16进制显示 AAH

(STACK 定义略)
DATA SEGMENT
X DB 0AAH
DATA ENDS

CODE SEGMENT
ASSUME CS:CODE, DS:DATA, SS:STACK
MAIN PROC FAR
MOV AX, DATA
MOV DS, AX; DS对准数据段
MOV CX, 8; 准备循环
NEXT:
ROL X, 1; 不带CF循环左移
MOV **DL**, X
AND DL, 1; 取左移前的最高位
ADD DL, 30H; 转为ASCII码
MOV AH, 2
INT 21H; 显示左移前的最高位
LOOP NEXT

MOV AH, 4CH
INT 21H; 退出
MAIN ENDP
CODE ENDS
END MAIN

MOV AX, DATA

MOV DS, AX; DS对准数据段

MOV **DL**, X

MOV CL, 4

SHR DL, CL; 逻辑右移, 处理原来的高四位, 高位补0

CMP DL, 9; 0—9, ASCII码加30H, A—F, ASCII码加37H

JBE NEXT1; 小于等于则转走, 无符号数比较

ADD DL, 7; A—F, ASCII码多加7

NEXT1:

ADD DL, 30H

MOV AH, 2

INT 21H

MOV DL,X; 准备处理低四位

AND DL,0FH; 保留低四位

CMP DL, 9

JBE NEXT2

ADD DL, 7

NEXT2:

ADD DL, 30H

MOV AH, 2

INT 21H

.....

扫码练习



大作业1

- Job 1

- 从键盘输入(回车符结束0DH), 从小到大排序并显示

- 定义数据段
 - 输入过程
 - 循环次数

- 进一步考虑: 如果输入‘\$’?

大作业1

- 数据段定义

```
DATA SEGMENT
```

```
    STR DB 100 DUP(?)
```

```
DATA ENDS
```

代码段中让DS对准数据段:

```
    MOV AX, DATA
```

```
    MOV DS, AX
```

SI: 寄存器间接寻址方式访问STR

```
    MOV SI, OFFSET STR; DS:SI逻辑寻址
```

程序框架:

```
STACK SEGMENT STACK
```

```
    ....
```

```
STACK ENDS
```

```
DATA SEGMENT
```

```
    ....
```

```
DATA ENDS
```

```
CODE SEGMENT
```

```
    ....
```

```
CODE ENDS
```

```
    END MAIN
```

大作业1

循环次数？

- 键盘输入代码

DEC SI
MOV BX, SI

INPUT: MOV AH, 1

INT 21H

MOV [SI], AL; SI为STR偏移地址，SI缺省搭配DS

INC SI; 地址增加，SI从0开始增加，正好为输入字符的个数

CMP AL, 0DH; 与回车符的ASCII码比较

JNE INPUT; 没有回车则继续输入

输入完成后放入结束字符：

DEC SI; 输入字符个数

MOV BYTE PTR [SI], '\$'

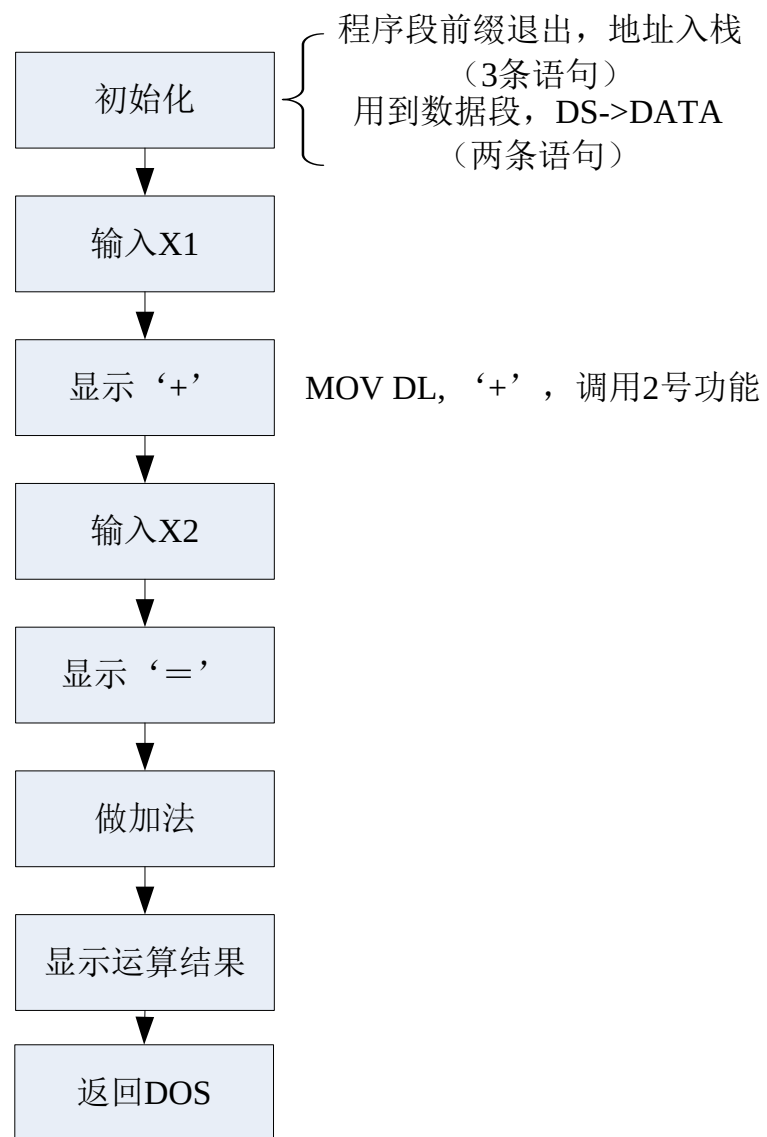
大作业1

- Job 2

- 从键盘输入2个四位十进制数(X1, X2)
- 相加得到X3并显示**十进制结果（BCD码相加）**
 - **例如：** $1234 + 5678 = \times \times \times \times \times$ ，相加后可能产生进位，得到一个五位十进制数
 - 简化运算：以**非压缩型BCD码**存储在数据段（ASCII码转非压缩型BCD）并进行十进制调整
 - 去掉起始的0

大作业1

- 数据段定义
- 输入子程序
- 输入 X1, X2
- 加法
- 结果显示



大作业1

- 数据段定义
非压缩型BCD码

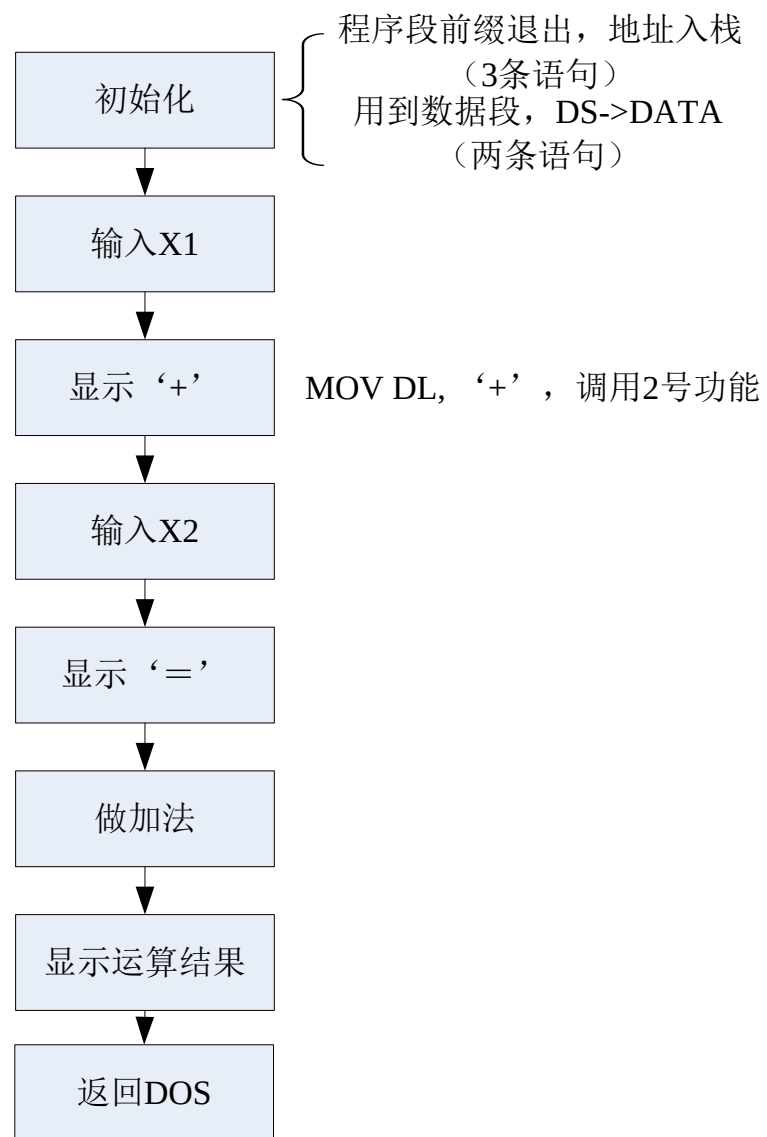
DATA SEGMENT

X1 DB 4 DUP(?)

X2 DB 4 DUP(?)

X3 DB 5 DUP(?)

DATA ENDS



大作业1

- 输入子程序（键入8个数字，调用8次）

代码段程序框架：

CODE SEGMENT

MAIN PROC

.....

MAIN ENDP

KEYIN PROC

.....

KEYIN ENDP

CODE ENDS

END MAIN

KEYIN PROC; 判断输入数字并回显
AGAIN:

MOV AH, 8

INT 21H;

CMP AL, 30H;

JB AGAIN; 小于返回，等待重新输入

CMP AL, 39H

JA AGAIN; 大于返回，等待重新输入

MOV DL, AL

MOV AH, 2

INT 21H; 为' 0'-'9'的ASCII码则显示

RET

KEYIN ENDP

大作业1

- 输入 X1, X2

MOV BX, OFFSET X1; X2同样操作

MOV CX, 4

NEXT:

CALL KEYIN; 输入字符的ASCII码存在AL中

AND AL, 0FH; 将AL中ASCII码转换对应BCD码

MOV [BX], AL

INC BX

LOOP NEXT

X1	01
	02
	03
	04
X2	05
	06
	07
	08
X3	

大作业1

- 加法

- BCD加法调整指令

MOV SI, OFFSET X1+3; X1个位

MOV DI, OFFSET X2+3; X2个位

MOV BX, OFFSET X3+4; X3个位

X1	01
	02
	03
	04
X2	05
	06
	07
	08
X3	

MOV CX, 4; 开始循环

OR CX, CX; CX不变, 标志位CF=0

NEXT2: MOV AL, [SI]

ADC AL, [DI]; 第一次CF=0

AAA; 加法调整

MOV [BX], AL

DEC SI

DEC DI

DEC BX

LOOP NEXT2

MOV AL, 0;

ADC AL, 0; 将千位相加的CF值取出

MOV [BX], AL

大作业1

- 结果显示

MOV BX, OFFSET X3

MOV CX, 5

NEXT3:

MOV DL, [BX]

ADD DL, 30H; 非压缩BCD转换为ASCII码

MOV AH, 2

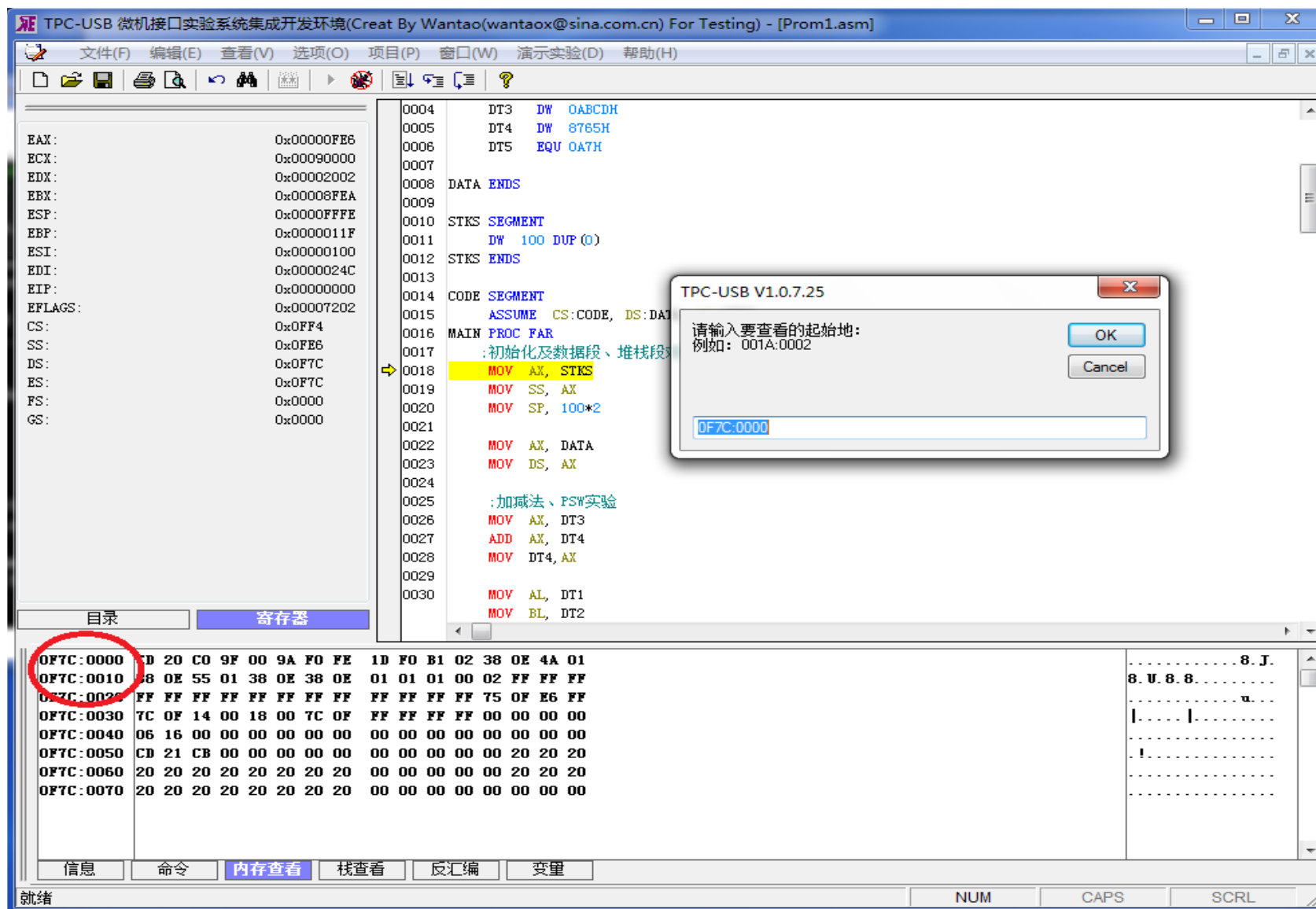
INT 21H

INC BX

LOOP NEXT3

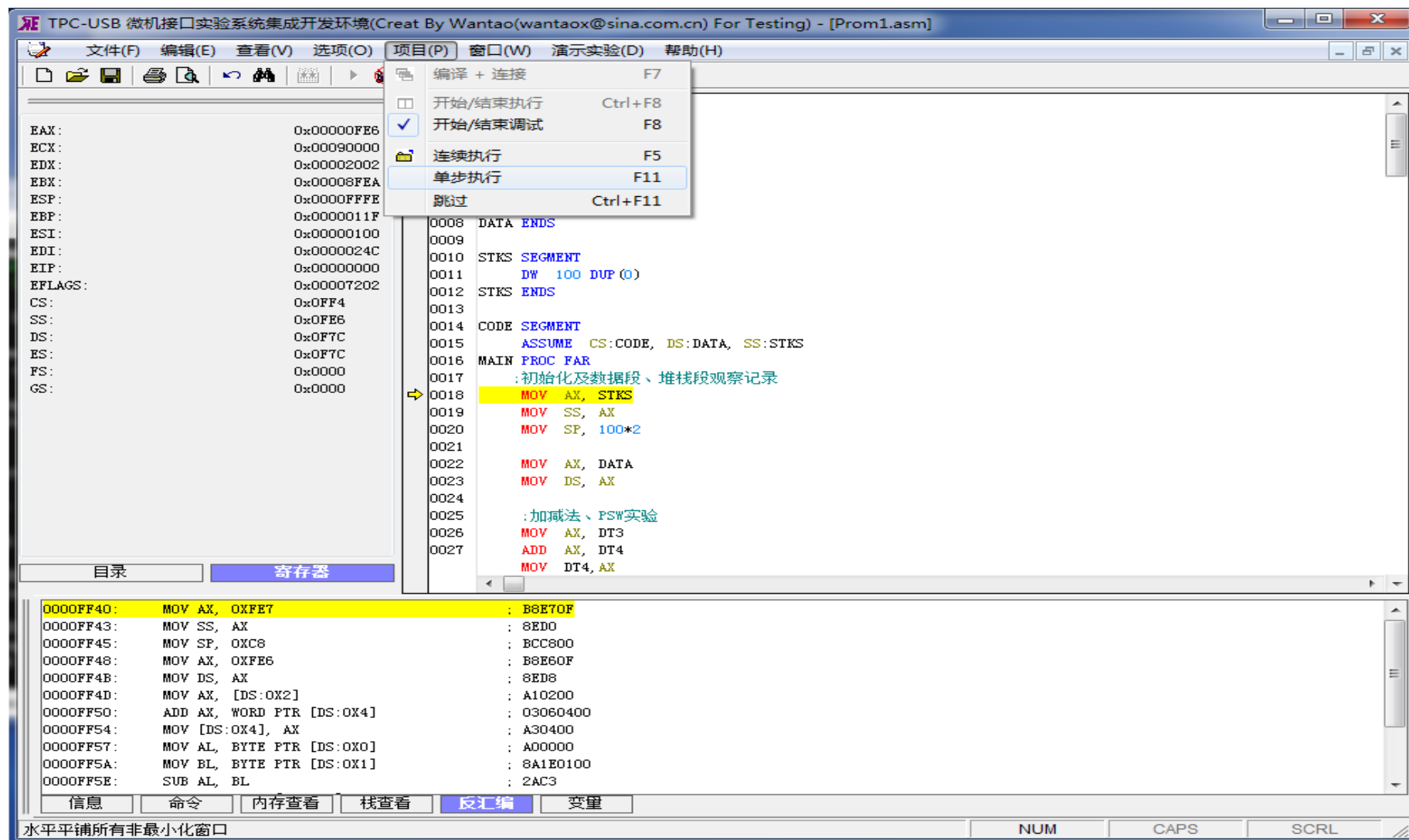
Debug

Debug调试/查看寄存器和内存



Debug

Debug调试/反汇编



Homework

1. 下列变量各占多少字节？↵

A1 DW 23H, 5678H↵

A2 DB 3 DUP(?), 0AH, 0DH, '\$'↵

A3 DD 5 DUP(1234H, 567890H)↵

A4 DB 4 DUP(3 DUP(1,2, 'ABC'))↵

↵

2. 有符号定义语句如下，L 的值为多少？↵

BUF DB 3,4,5, '123'↵

ABUF DB 0↵

L EQU ABUF-BUF↵

↵

3. 假设程序中的数据定义如下，PLENTH 的值为多少？表示什么意义？↵

PAR DW ?↵

PNAME DB 16 DUP(?)↵

COUNT DD ?↵

PLENTH EQU \$-PAR↵

Homework

4. 下面各 MOV 指令执行后, AX, BX, CX 寄存器的值是多少?

DA1 DB ?_↵

DA2 DW 10 DUP(?)_↵

DA3 DB 'ABCD'_↵

MOV AX, TYPE DA1_↵

MOV BX, SIZE DA2_↵

MOV CX, LENGTH DA3_↵

_↵

5. 以下程序完成后, AH 等于多少? _↵

IN AL, 5FH_↵

TEST AL, 80H_↵

JZ L1_↵

MOV AH, 0_↵

JMP STOP_↵

L1: MOV AH, 0FFH_↵

STOP: HALT_↵

Review of chapter 4

- 汇编程序: 指令+伪指令(format)
- 伪指令Pseudo-instructions: 数据定义, 表达式定义, 段分配, 段声明, 子程序定义, 程序结束
- 汇编编程: text writing, assembly(MASM.EXE), link(LINK.EXE), execution and Debug(-r, -d, -e, -u, -g, -t, -p) , PSP
- DOS 功能调用(INT 21H function 1、 2、 6、 8、 9、 4CH)
- Assembly language programming